Project Overview — Simple Explanation

A plain-language tour of the whole project: what it is, how the pieces fit together (the new architecture), and how data flows through it. For step-by-step setup see INSTALL.md; for a deep feature walkthrough see DEMONSTRATION.md.

---

1. What is this project?

It's a **simulator for "fog computing"** task scheduling, with an **AI-style scheduler** that decides where each task should run.

- **IoT devices** (healthcare monitors, traffic sensors, industrial controllers) constantly produce small jobs called **tasks**.
- Each task can run on a nearby **fog node** (fast, but limited power) or on the distant **cloud** (unlimited power, but slower).
- A **scheduler** decides, for every task, *where it should go* — balancing speed, energy, and load.

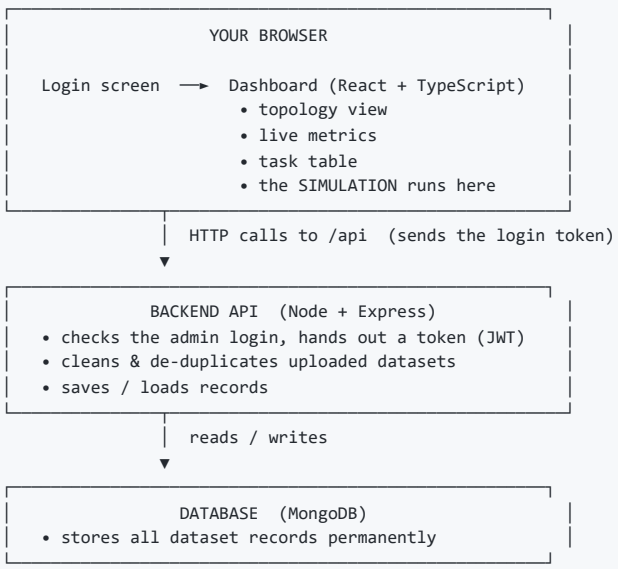
The app lets you **load real or synthetic data, run the scheduler live, and watch what happens** — latency, energy use, how busy each fog node is — all on a dashboard.

> Think of it like an air-traffic controller, but for computing jobs: lots of planes (tasks) need to land somewhere (a fog node or the cloud), and the controller (scheduler) picks the best runway for each one.

---

2. The new architecture (big picture)

The project used to be a single browser app. It now has **three parts** plus the simulation that runs in your browser:



What each part does, in plain words:

Part	Tech	Its job
Frontend	React, TypeScript, Vite, Tailwind	Everything you see and click. Also *runs the simulation* (the animation and the math happen in your browser).
Backend API	Node.js, Express	The gatekeeper and the librarian: it checks your login and reads/writes data. The browser never talks to the database directly.
Database	MongoDB	Long-term memory. Datasets you load are stored here so they survive a page reload.
Simulation engine	runs in the browser	The "controller" that moves tasks to fog nodes / cloud each second and computes the metrics.

A small **Vite proxy** forwards anything the browser requests at `/api` to the backend, so the two run side-by-side during development.

---

3. How data flows (a typical session)

```
1. LOG IN
   You type admin / admin123  →  Backend checks it  →  returns a token.
   The browser keeps the token and shows the dashboard.

2. LOAD A DATASET
   You click "Datasets" and pick Sample / Synthetic / your CSV.
   Browser → Backend: "here are these rows."
   Backend cleans them (fixes types, drops bad rows) and REMOVES DUPLICATES,
   then stores the unique records in MongoDB and reports how many were saved.

3. BUILD THE FOG NODES (automatic)
   The app looks at how many records there are and creates the right number of
   fog nodes:  1 node per 1,000 records, kept between 3 and 12.
   (10,000 records  →  10 fog nodes.)

4. PICK A STRATEGY  &  PRESS START
   You choose how the scheduler thinks (see §4) and start the simulation.
   Every second the engine: generates tasks → the scheduler assigns each one to a
   fog node or the cloud → tasks "process" and complete → the dashboard updates.

5. WATCH
   • Topology  – tasks flowing from devices to nodes
   • Metrics   – latency, energy, utilization, cloud offload (all live)
   • Task table – every task with its status, CPU, memory, latency, energy, node

6. RELOAD-PROOF
   Refresh the page and your saved dataset comes back from MongoDB.
```

---

4. The scheduler — four ways to decide

When a task appears, the scheduler picks where it runs. You can switch the "brain":

Strategy	How it chooses
DRL Optimized *(default)*	A balanced score combining free resources, closeness, energy, and load — multiplied by the task's priority. Best all-round.
Priority Aware	Urgent tasks (critical/high) go to the closest, fastest node first.
Load Balanced	Sends each task to the *least busy* fog node.
Energy Efficient	Sends each task to the node using the *least energy*.

If no fog node has room, the task is **offloaded to the cloud** (more capacity, higher latency).

---

5. The seven things it does

1. **Admin login** – the whole dashboard is locked until you sign in.
2. **MongoDB storage** – datasets are saved permanently and restored on reload.
3. **Automatic fog nodes** – node count scales to the dataset size (3–12).
4. **Task table** – every task listed with full details and units (CPU %, MB, ms, kW).
5. **Live metrics & node status** – fixed so they update continuously while running.
6. **Clean datasets** – validation + duplicate removal produce a unique, tidy dataset.
7. **Docs** – this overview, plus the install and demonstration guides.

---

6. Key words (glossary)

- **IoT device** – a sensor/machine that produces tasks (healthcare, traffic, industrial).
- **Task** – one unit of work, with CPU %, memory (MB), and a target latency (ms).
- **Fog node** – a nearby computer with limited capacity; fast but can fill up.
- **Cloud** – a far-away data center; effectively unlimited but slower.
- **Scheduler** – the logic that assigns each task to a fog node or the cloud.
- **Latency (ms)** – how long a task takes; lower is better.
- **Energy (kW)** – power a node uses; modeled as `cpu × 0.5 + memory × 0.1`.
- **Utilization (%)** – how full the fog nodes are.
- **Cloud offload (%)** – share of tasks sent to the cloud instead of a fog node.

---

7. Where things live (project map)

```
Al-Fog-computing/
├─ run.bat                    one-click install + run
├─ scripts/                   setup.bat, start.bat, clean-data.bat
├─ src/                       FRONTEND (React app + the simulation)
│  ├─ App.tsx                 the dashboard + login gate
│  ├─ hooks/useSimulation.ts  the simulation engine
│  ├─ utils/                  scheduler logic, fog-node creation, data loading
│  ├─ components/             topology, metrics, task table, dataset UI, login
│  ├─ context/AuthContext     login state
│  └─ api/client.ts           talks to the backend
├─ server/                    BACKEND API (Express + MongoDB)
│  └─ src/{routes,models,lib,middleware}
├─ data/clean/                cleaned dataset output
└─ docs/                      OVERVIEW (this) · INSTALL · DEMONSTRATION
```

---

8. In one sentence

> A login-protected dashboard where you load a dataset (cleaned and stored in MongoDB), the app automatically builds the right number of fog nodes, and an AI-style scheduler distributes tasks across those nodes and the cloud while you watch the latency, energy, and load update live.